TECHNICAL DESIGN DOCUMENT

面向猎头的“根据 JD 寻找匹配简历” AI-Native 工程技术设计文档（TDD）

对应 PRD v1.0；覆盖业务主系统、Agent 运维监控平台、Agent Evals 平台、并行 Agent 开发 Harness

版本	v1.0	状态	评审版
阶段	MVP + Day-0 工程骨架	日期	2026-03-23
技术基线	Angular / Python / PostgreSQL / Temporal / PydanticAI / Sealos	文档属性	可评审 / 可交付 / 可追责
输入基线	PRD v1.0、AI-Native Harness 方法论、深度研究报告	目标读者	架构 / 后端 / 前端 / 测试 / 运维 / AI 平台

语义先于代码 OWL + SHACL 不是装饰层，而是所有概念、字段和约束的第一来源。	**合同先于实现** OpenAPI / AsyncAPI 先改后用；前后端与 worker 不能靠口头同步。
闸门先于并行 在允许多个 agent 同时写代码之前，必须先有导入边界、codegen diff、eval gate。	**Ops / Evals 同起步** 运维监控与评测不是“以后补”，而是 AI-Native 工程最早的系统组成部分。

本文档只定义 “怎么做” 的技术边界、实现方式、接口契约、数据模型、运行与评测体系、闸门策略与架构取舍；不复述 PRD 的业务价值论证。

0. 文档导航

这份 TDD 的目标不是把技术细节写成流水账，而是让前端、后端、测试、运维与多 agent 并行开发在没有额外口头解释的情况下也能对齐。

序号	章节
1	先回答核心问题：空仓库一开始到底先建什么
2	设计基线与范围（含新增 Ops / Evals 平台）
3	总体技术决策与 ADR
4	Monorepo、文档体系、Obsidian 联动与 Harness 治理
5	边界上下文、部署单元与运行架构
6	关键 API / 事件契约
7	工作流与控制流（Temporal）
8	数据模型、索引、幂等与 Postgres 策略
9	前端、Agent Ops、Agent Evals 平台设计
10	CI/CD、质量闸门、自建 runner 与安全
11	非功能实现、降级、审计与合规
12	实施节奏、Day 0 清单、风险与待确认问题
A	附录：错误码、Repo 树、追踪矩阵、参考资料

关于 OpenAI Harness Engineering 文章怎么用

可以借鉴它的工作原则：docs/ 作为系统事实源、AGENTS.md 作为目录而不是大百科、执行计划入仓、文档新鲜度用 CI 机械校验；但不建议原样照抄目录树。你的仓库必须显式容纳 ontology、contracts、agent-governance、observability、evals 与 docs/_generated 这些 AI-Native 特有边界。

1. 先回答核心问题：空仓库一开始到底先建什么

一句话：不是先让 agent 写业务代码，也不是只先摆一个目录树；而是先建立“仓库操作系统”。所谓仓库操作系统，是一组可执行的边界、合同、索引、闸门和审计规则。没有它，后续的多 agent 并行只会把仓库做成高产但不可维护的随机森林。

在你的约束下，第一周最重要的工作不是“把 MVP 页面做出来”，而是先把以下七件事同时落到仓库里：1）目录即边界；2）ontology / contracts 成为 SSOT；3）关键导入规则可测试；4）agent 只写允许目录；5）docs 成为事实源并与 Obsidian 同步；6）Ops / Evals 有最小骨架；7）CI 能一键拒绝越界与漂移。

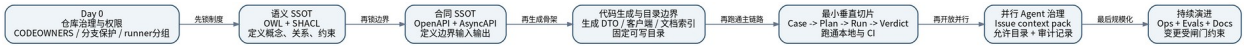

图 1 空仓库起步的正确顺序：先锁制度，再生成骨架，再开放并行。

1.1 不是“先建目录”，而是“先建可执行边界”

必须先有的东西	为什么必须当天完成	如果缺失会发生什么
CODEOWNERS + 分支保护 + runner 分组	先锁谁能改什么、哪些工作流能在哪类 runner 上运行	agent 可以在无保护的主干上随意放大错误；自建 runner 也会暴露更大攻击面
ontology/core + ontology/shapes	从第一天开始就定义统一概念、关系和可验证约束	AI 会自己创造字段和概念，语义漂移会比代码漂移更快
contracts/openapi + contracts/asyncapi	前后端、worker、测试与 QA 可立即并行	实现先跑出去后再补接口文档，后续只能靠人工猜
make validate + CI skeleton	把“制度”变成任何 PR 都必须通过的闸门	目录规则和文档规则会退化成口头约定
issues/context-pack + agents/runs	把 issue-centric 并行工作变成可审计、可收敛的工程流程	多个 agent 将读取全仓库噪声上下文，产出更发散
docs/ 事实源 + docs/_generated	把架构和知识图谱与代码绑定，避免 Obsidian 另起炉灶	你会很快拥有两套文档：一套给人看，一套给 agent 猜
Ops / Evals 最小骨架	AI-Native 项目如果没有运行可观测与评测，后续只能盲修	系统出了问题以后，你只能通过日志碎片猜哪里慢、哪里错、模型是否退化

1.2 我对你的直接建议

> **强观点**
> 你应该把 docs/ 直接当作 Obsidian vault 的内容根，而不是再维护一个独立的 obsidian-vault 内容源。若团队确实要保留单独目录，也只能放自动生成镜像或本地配置，不允许手工改写双份知识。否则“可视化知识图谱”会在三周内演化成“第二套过时文档系统”。

- 前端不要一开始拆成四个独立可部署应用。先用一个 Angular shell 承载 /cases、/agents、/ops、/evals 四个路由域，再用 libs/ts 把数据访问与 UI 边界切开。真正需要独立发布或独立团队 ownership 时，再拆应用。

- 后端不要一开始上微服务。先做“模块化单体 + 独立 worker 进程 + 明确边界上下文 + 独立 schema / 包边界”。这更符合 MVP 和 Postgres-first，也更适合 AI 理解。
- Postgres-only 要坚持，但不要教条。结构化业务数据、投影、outbox、评测结果都放 Postgres；真正的大体积导出文件和截图类工件，允许作为后续例外使用对象存储。
- Temporal 只用于长流程、可重试、可恢复的编排，不要把普通 CRUD 都包成 Workflow。否则复杂度会无意义上升。
- PydanticAI 只出现在 Activity 或离线 eval runner 中，不允许放进需要确定性的 Temporal Workflow 代码。

1.3 OpenAI Harness 文章的目录结构能不能用

结论	建议
原则上可以借鉴；目录树不要照抄	借鉴 docs 作为系统事实源、AGENTS.md 只做入口、执行计划入仓、CI 校验文档新鲜度；但你的仓库必须额外纳入 ontology、contracts、agent-governance、evals、observability、docs/_generated 与 issue context pack。
AGENTS.md 要短	它应该只有仓库地图、关键规则和指向深层文档的链接；不要让它变成 1000 行大手册。
计划必须成为一等工件	跨上下文或超过 1 天的工作强制有 exec plan，完成后归档到 completed；技术债也要留在仓库里，而不是散落在聊天工具。
文档必须被机械校验	目录索引、交叉链接、生成物 diff、过时文档扫描都要进 CI，不接受“大家自觉维护”。

2. 设计基线与范围（含新增 Ops / Evals 平台）

本 TDD 以 PRD v1.0 为业务边界，以 AI-Native Harness 方法论为工程边界。与通常把运维与评测留到二期不同，这里把 Agent Ops 平台与 Agent Evals 平台上升为系统基线：它们不是附属页面，而是为了让并行 agent 开发可控、让 AI 辅助质量可测、让后续百万行代码仍可演进的基础设施。

范围	本期纳入	本期不纳入
业务主系统	Case / JD Version / Keyword Draft / Condition Plan / Search Run / Page Snapshot / Candidate Review / Verdict / Pool / Export / Audit	黑箱推荐首页、自建向量检索、自动触达候选人、自动提交最终 verdict
Agent Ops 平台	任务队列、失败分类、超时分类、版本指标、工作流健康、关键链路耗时、故障 drill-down	全量替代 Grafana / Temporal UI 的底层观测能力
Agent Evals 平台	数据集、样本、评测套件、版本对	完整 MLOps 实验跟踪、自动超参搜

范围	本期纳入	本期不纳入
	比、阻断型阈值、样本 diff	索
工程治理	issue context pack、agent registry、playbook、runs 审计、docs sync、codegen、架构测试、critical eval gate	自由格式的 agent 任意写仓库、纯人工口头交接

2.1 设计目标

- 从 Day 0 起，把仓库设计成“对 AI legible、对人可追责”，而不是只对熟悉上下文的人可用。
- 保证顾问主流程在 AI 异常时仍然可用，运维团队能看见失败原因，算法/平台团队能量化 AI 版本变化。
- 在不引入过多基础设施的前提下，让 PostgreSQL 成为事实源、Temporal 成为编排层、Contracts 成为边界层、Docs 成为知识层。
- 保证前端、后端、QA、运维、AI 平台和多个 code agent 可以围绕同一组契约独立推进。

2.2 关键前提与约束

约束	设计含义
代码长期扩展到百万行	必须把边界上下文、导入规则、文档索引和变更治理放在 Day 0，而不是靠后期“代码重构一遍”。
主要编程由并行 code agent 完成	一切默认不可信；必须先把允许目录、合同、输出 schema、审计记录与 CI gate 准备好。
前端 Angular / 后端 Python	双语言合同生成与共享类型是基础能力，不允许同一 DTO 手写两份。
Postgres-only as much as possible	MVP 不引入 Redis / Kafka / Elastic；优先使用 Postgres 投影、JSONB、GIN / trigram、outbox。
部署到 Sealos，代码托管 GitHub 私有仓库，自建 CI runner	必须设计 runner 分组、最小权限、环境隔离与私有仓库策略；构建与部署 runner 不能混用。
Obsidian 需要展示知识图谱	文档必须仓库内版本化，且生成 Markdown 索引要能回链到 ontology / contracts / code / dashboards。

3. 总体技术决策与 ADR

本节只回答技术实现与取舍，不重复 PRD 里的业务理由。每个 ADR 都要回答两件事：选什么，以及为什么暂时不选另一个看上去更“强大”的方案。

ADR	本期采用	明确不采用	核心原因	后续重评触发
ADR-001	模块化单体 + 独立 worker 进程	Day 0 微服务拆分	MVP 团队规模、复杂度与 Postgres-first 更适合单仓单体边界；真正需要拆服务时再按上下文抽取	单体部署/测试时间失控；跨域团队独立发布成为刚需
ADR-002	单一 Angular shell + 路由域隔离	四个独立前端应用 / 微前端	初期避免重复脚手架、重复认证、重复数据访问层；仍保留未来拆分空间	不同控制台发布节奏明显分离
ADR-003	Postgres 写模型 + 读投影 + outbox	Redis + Kafka + Elastic 全家桶	先满足 PRD 与 Ops / Evals 的最小闭环，避免基础设施先于问题	查询延迟或吞吐持续超出阈值
ADR-004	Temporal 只编排长流程	所有请求都进 Workflow	普通同步 CRUD 不需要 Workflow 语义；滥用会让调试、测试与心智负担上升	业务大量转为长流程、跨上下文补偿变复杂
ADR-005	docs/ 即知识事实源，兼作 Obsidian vault 内容根	docs 与 obsidian-vault 双源手工维护	双源文档一定会漂移；需要一个可校验的唯一源	未来确需对外公开文档镜像
ADR-006	合同生成 DTO / 客户端，手工只写领域行为	边界模型手工双写	多语言双写最容易在并行 agent 环境下漂移	生成器长期不稳定且阻塞交付
ADR-007	Ops / Evals 从 Day 0 建最小骨架	二期再补	AI-Native 项目如果没有质量闭环，后续改动无法判断是变好还是变坏	无
ADR-008	自建 runner 分组 + 尽量临时化 / 一次性	持久化共用 runner 池随意复用	私有仓库也存在 fork / PR 代码污染风险，持久 runner 容易泄漏上下文与凭据	若后续切回 GitHub-hosted 为主

3.1 关键架构原则

- 领域行为只允许依赖领域内抽象；domain 不得直接碰数据库、HTTP、Temporal SDK client 或日志实现。
- 同步边界用 OpenAPI，异步边界用 AsyncAPI；领域事件与集成事件分开命名与版本管理。
- 所有 AI 输出都必须命中明确的输出 schema，并记录 prompt_version、model_version、input_hash、output_schema_version。
- 对外部简历库的每次页抓取都必须快照化；任何“重新请求一次看看”都不能覆盖历史事实。
- 导出脱敏、列表脱敏、详情脱敏、看板脱敏使用同一套字段策略，不允许四套逻辑各写各的。

- 评测平台不只比较“模型好不好”，还必须比较“是否合规、是否泄露、是否正确降级”。

4. Monorepo、文档体系、Obsidian 联动与 Harness 治理

这个项目最重要的技术资产不是某个 Python 包或 Angular 组件，而是仓库本身的可导航性。对并行 agent 来说，任何不在仓库里的信息都等于不存在；因此目录树、文档树、context pack、生成物和审计记录必须一起设计。

图 2 Issue-centric 并行开发的最小闭环：issue -> context pack -> agent PR -> 闸门 -> 合并。

4.1 推荐 Monorepo 目录

```
repo-root/
├── AGENTS.md
├── README.md
├── Makefile
├── .github/
│   └── workflows/
├── apps/
│   └── web-console/                # Angular shell (/cases /agents /ops /evals)
├── services/
│   ├── platform-api/               # FastAPI 模块化单体
│   ├── temporal-worker/            # Temporal worker + Activities
│   └── eval-runner/                # 行器（可用 worker 包）
├── libs/
│   ├── py/
│   │   ├── domain-kernel/
│   │   ├── contracts-generated/
│   │   └── testkit/
│   └── ts/
│       ├── api-client-generated/
│       ├── feature-shared/
│       └── ui-shared/
├── ontology/
│   ├── core/
│   ├── shapes/
│   ├── examples/
│   └── tooling/
├── contracts/
│   ├── openapi/
│   ├── asyncapi/
│   └── jsonschema/
├── prompts/
│   ├── system/
│   └── patterns/
├── issues/
│   ├── templates/
│   └── context-packs/
├── agents/
│   ├── registry/
│   ├── playbooks/
│   └── runs/
├── tools/
│   ├── bootstrap/
```

```
│   ├── codegen/
│   ├── ci/
│   └── docs-sync/
└── docs/
    ├── 00-INDEX.md
    ├── ARCHITECTURE/
    ├── ADR/
    ├── EXEC-PLANS/
    │   ├── active/
    │   ├── completed/
    │   └── tech-debt.md
    ├── PRODUCT/
    ├── ONTOLOGY/
    ├── CONTRACTS/
    ├── AGENT-GOVERNANCE/
    ├── OBSERVABILITY/
    ├── EVALS/
    ├── SECURITY/
    ├── RUNBOOKS/
    ├── QUALITY/
    ├── REFERENCES/
    └── _generated/                 # 由 codegen / docs-sync 自动生成
```

4.2 文档目录的职责划分

目录	内容性质	是否允许手工修改	强制校验
AGENTS.md	入口文档；仓库地图、最关键约束、如何定位深层文档	允许，但必须短小	长度上限、链接有效性
docs/ARCHITECTURE	系统图、上下文地图、部署视图、层级规则	允许	链接有效性、与代码路径存在性
docs/ADR	技术决策与取舍记录	允许	模板完整性、编号唯一性
docs/EXEC-PLANS	复杂任务执行计划、进度、决策日志、技术债	允许	状态迁移、过期计划提醒
docs/ONTOLOGY	建模规范、版本规则、概念索引	允许	与 ontology/ 版本对齐
docs/CONTRACTS	兼容策略、字段规范、错误码约定	允许	与 contracts/ 版本对齐
docs/OBSERVABILITY	指标词典、告警策略、dashboard 索引、runbook	允许	dashboard 引用存在性
docs/EVALS	suite 说明、评分口径、阈值、baseline 版本	允许	阈值文件存在性、评测结果索引
docs/_generated	数据库 schema 索引、接口索引、知识图谱节点页	禁止手改	生成后 diff 必须为 0

4.3 Obsidian 联动策略

- 默认选择：docs/ 内容目录直接作为 Obsidian 可视化根，知识图谱的节点来自 docs/ 手工文档与 docs/_generated 自动文档。
- 每个概念、事件、接口、关键表、告警策略都生成一页 Markdown；frontmatter 至少包含 sourcePath、ontologyVersion、contractVersion、owner、generatedAt。
- Obsidian 中允许沉淀“建模讨论 / 架构注释 / 领域澄清”，但必须通过 agent 转成 PR 对应的文档或本体变更，不能停留在孤立笔记。
- 本地工作区偏好（例如 pane 布局）默认不入仓；共享的 graph 配置、核心模板和索引脚本入仓。

> **关键规则**
> 如果一个知识点既影响 agent 决策又影响系统实现，那它必须变成仓库内可检索、可版本化、可校验的 Markdown，而不是留在聊天记录里。

4.4 并行 agent 的治理模型

工件	最小要求
Issue 模板	必须记录目标、影响 PRD / AC 编号、允许目录、依赖合同、风险与回滚方式
Context Pack	只注入当前任务相关的 ontology 片段、contracts 片段、架构规则、测试要求和目标版本，不把整仓库生吞给 agent
Agent Registry	每类 agent 声明能力、可写目录、禁改目录、默认 playbook、需要通过的闸门
Agent Run 审计	必须记录 prompt hash、输入上下文摘要、写入路径、验证结果、失败原因、关联 PR
Doc Gardener	定时扫描过时文档与断链，自动提修复 PR；不是靠人偶尔顺手整理

5. 边界上下文、部署单元与运行架构

本系统不在 Day 0 拆成多个微服务，但必须在代码、数据库 schema、合同和文档上先把边界上下文切清楚。这样做的目的不是“讲 DDD 的道理”，而是让 agent 无法跨边界乱写。

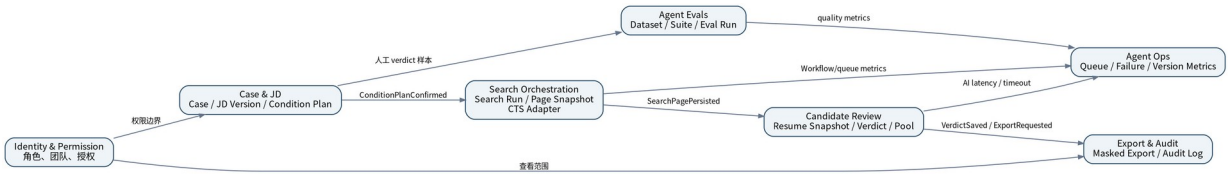

图 3 边界上下文：业务系统、Ops、Evals 与权限/审计的关系。

5.1 边界上下文清单

上下文	核心职责	写模型 / 事实源	主要读投影	集成事件
Identity & Permission	用户、团队、角色、可见范围	user_account / team / role_binding	当前用户权限投影	UserRoleChanged
Case & JD	Case、JD Version、关键词草案、Condition Plan	jd_case / jd_version / keyword_draft_job / condition_plan	case_list_view	ConditionPlanConfirmed
Search Orchestration	Search Run、分页抓取、页面快照、上游容错	search_run / search_run_page / external_candidate_identity	run_list_view / run_progress_view	SearchRunStarted / SearchPagePersisted / SearchRunCompleted
Candidate Review	简历快照、AI 摘录、verdict、候选池	case_candidate / resume_snapshot / verdict_history / pool_entry	candidate_detail_view / pool_view	CandidateVerdicted
Export & Audit	导出任务、脱敏、敏感动作审计	export_job / audit_log	export_status_view / audit_query_view	ExportRequested / ExportCompleted
Agent Ops	队列、失败、超时、版本指标、workflow 健康	workflow_health / ai_version_metrics / agent_run	ops_summary_view	WorkflowHealthChanged
Agent Evals	数据集、套件、评测运行、结果汇总	eval_dataset / eval_sample / eval_run / eval_result	eval_compare_view	EvalRunCompleted

5.2 部署单元

部署单元	职责	伸缩方式	说明
apps/web-console	顾问工作台 + Agent Ops + Agent Evals 前端	按前端标准水平扩容	一个 shell 应用即可；通过路由域实现逻辑边界
services/platform-api	同步 API、权限、投影查询、导出发起、评测管理 API	无状态水平扩容	模块化单体，内部按上下文划包
services/temporal-worker	SearchRun / KeywordDraft / ResumeAnalysis / Export / Eval 的 Activities	按任务队列与并发配置扩容	与 API 分离，避免长任务拖垮同步请求
Temporal Server	Workflow 状态与调度	独立部署	使用 PostgreSQL 持久化
PostgreSQL	写模型、读投影、outbox、	纵向优先，必要时读写分离	MVP 的事实源

部署单元	职责	伸缩方式	说明
	评测结果、审计		
OTel + Grafana Stack	metrics / logs / traces / dashboards / runbooks	独立运维	既服务人，也服务 agent

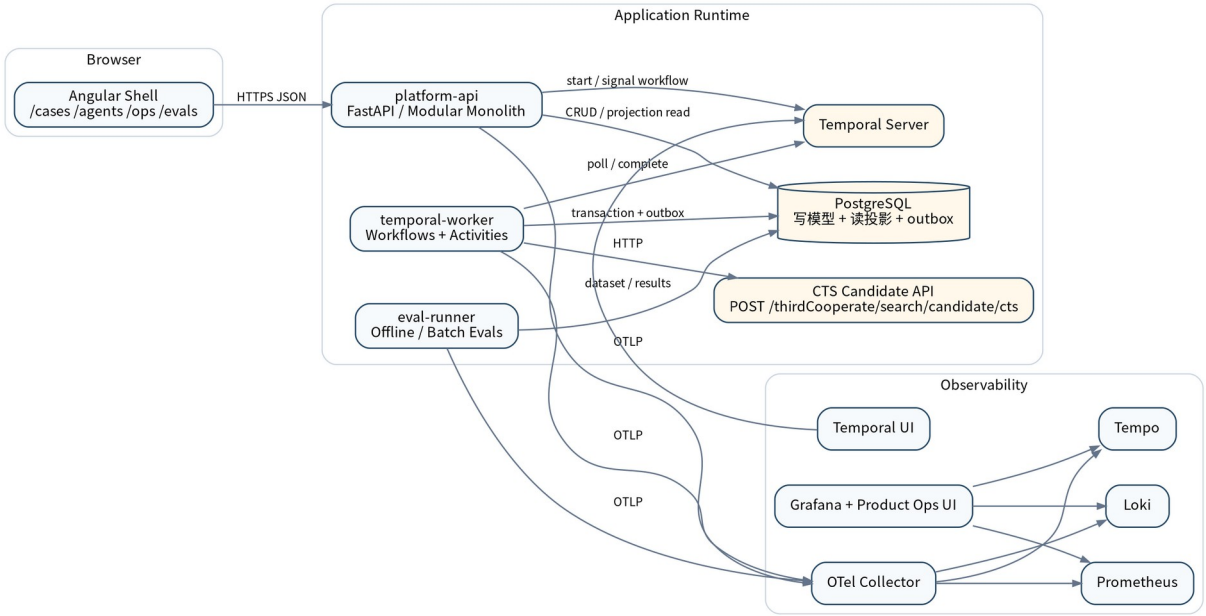

图 4 运行架构：一个 Angular shell + 一个 API + 一个 worker + 一套 Postgres/Temporal/Observability。

5.3 本期选型落地

领域	采用	说明
HTTP 框架	FastAPI	OpenAPI 先天友好，与 Pydantic 边界模型配合紧密，适合作为合同优先的 Python API 层
领域实现	显式 domain / app / infra 分层	保持 domain 纯净，application 组织用例，infra 适配数据库/Temporal/外部 API
数据库访问	SQLAlchemy 2 + Alembic（显式模式）	选显式、可读、可迁移，不依赖太多魔法
前端	Angular shell + feature libraries	避免 Day 0 微前端复杂度
异步编排	Temporal	只用于长流程；Workflow 只编排，Activities 才做 I/O 与 LLM
Agent 框架	PydanticAI	用于 Activity 和离线 eval，不进入确定性 Workflow

领域	采用	说明
消息中间件	MVP 不引入外部 MQ	用 PostgreSQL outbox + poller 保证可靠发布；必要时后续替换

6. 关键 API / 事件契约

本节锁定前后端、worker 与外部系统的接口边界。前端可以基于这些契约先做 mock，QA 可以据此准备自动化用例，后端与 worker 只在这些合同允许的范围内实现。

6.1 外部 CTS 简历库契约

项	定义
上游接口	POST https://link.hewa.cn/thirdCooperate/search/candidate/cts
必需请求头	trace_id、tenant_key、tenant_secret
关键请求体字段	jd、keyword、company、position、location、degree、workExperienceRange、page、pageSize
特殊行为	page / pageSize 缺失、0 或非法值时可能返回 data: null；不能把它当作真实 0 结果
本系统适配规则	请求前强校验 + 响应后强判定 + 所有页级原始响应快照落库

6.2 内部 REST 契约（摘要）

Endpoint	Method	用途	关键入参	返回
/api/v1/cases	POST	创建 Case	title, ownerTeamId	caseId, status
/api/v1/cases/{caseId}/jd-versions	POST	创建 JD 版本	rawText, source	jdVersionId, versionNo
/api/v1/cases/{caseId}/keyword-draft-jobs	POST	发起关键词草案生成	jdVersionId, modelVersion, promptVersion	jobId, status=queued
/api/v1/condition-plans/{planId}:confirm	POST	确认检索方案	normalizedQuery, confirmedBy	planId, status=confirmed
/api/v1/search-runs	POST	创建 Search Run	caseId, planId, pageBudget,	runId, status=queued

Endpoint	Method	用途	关键入参	返回
			idempotencyKey	
/api/v1/search-runs/{runId}	GET	查看 Run 状态	-	status, progress, pageCount, errorSummary
/api/v1/search-runs/{runId}/pages	GET	查看已落库页快照	pageNo?	snapshot list
/api/v1/case-candidates/{id}	GET	查看候选详情	-	resumeSnapshot, aiAnalysis, verdictHistory
/api/v1/case-candidates/{id}/verdict	PUT	保存 verdict	verdict, reasons, notes, resumeSnapshotId	latestVerdict, updatedAt
/api/v1/exports	POST	发起导出	caseId, maskPolicy, reason, idempotencyKey	exportJobId, status=queued
/api/v1/ops/summary	GET	Ops 首页	timeRange?	queue, failures, latency, version metrics
/api/v1/evals/runs	POST	发起评测	suiteId, datasetId, targetVersion	evalRunId, status=queued

6.3 核心资源对象

对象	关键字段	说明
ConditionPlan	id, caseId, jdVersionId, mustTerms[], shouldTerms[], excludeTerms[], structuredFilters, evidenceRefs[]	用户确认后的检索方案；这是 Search Run 的唯一合法输入
SearchRun	id, caseId, planId, status, pageBudget, pagesCompleted, startedAt, finishedAt, errorCode, errorMessage	一次冻结的检索执行；启动后参数不可被原地改写
SearchRunPageSnapshot	id, runId, pageNo, upstreamRequest, upstreamResponse, normalizedCards[], fetchedAt, status	每页都必须快照化，用于复盘、幂等与审计
CaseCandidate	id, caseId, externalIdentityId, latestResumeSnapshotId, latestVerdict, dedupeStatus	Case 视角下的候选聚合实体
ResumeAnalysis	jobId, resumeSnapshotId,	AI 辅助信息，不得直接提交最终

对象	关键字段	说明
	modelVersion, promptVersion, summary, evidenceSpans[], riskFlags[], status	verdict
EvalRun	id, suiteId, datasetId, targetVersion, baselineVersion, status, summaryMetrics, blockingResult	评测平台的最小比较单位

6.4 关键事件契约（AsyncAPI）

事件名	触发时机	必含字段	消费者
condition-plan.confirmed.v1	Condition Plan 被确认	eventId, caseId, planId, jdVersionId, actorId, occurredAt	Search Orchestration
search-run.started.v1	Search Run 进入运行	eventId, runId, caseId, planId, pageBudget, occurredAt	Agent Ops
search-page.persisted.v1	某页快照落库成功	eventId, runId, pageNo, cardCount, occurredAt	Candidate Review, Agent Ops
candidate.verdicted.v1	用户提交 verdict	eventId, caseCandidateId, caseId, verdict, actorId, occurredAt	Audit, Evals
export.requested.v1	导出任务创建	eventId, exportJobId, caseId, maskPolicy, requestedBy, occurredAt	Export Worker, Audit
eval-run.completed.v1	评测完成	eventId, evalRunId, suiteId, blockingResult, summaryMetrics, occurredAt	Agent Ops, QUALITY docs

6.5 错误码与前端行为

HTTP	错误码	含义	前端处理
400	INVALID_PAGINATION_PARAMS	提交到 Search Run 的 page / pageBudget 非法	阻断提交，表单直接提示
401	UNAUTHENTICATED	未登录或会话失效	跳转登录
403	NO_CONTACT_PERMISSIO	无权限查看联系方式或发起	展示拒绝信息，但不得透出

HTTP	错误码	含义	前端处理
	N	敏感导出	局部敏感值
404	CASE_OR_CANDIDATE_NOT_FOUND	资源不存在或不在可见范围	标准 404
409	IDEMPOTENCY_CONFLICT	相同 idempotency key 请求重复	返回已有资源，不重复创建
422	PLAN_NOT_CONFIRMED	未确认的 Condition Plan 不能启动 Search Run	引导先确认条件
422	JD_TOO_LONG	JD 超过 AI 可处理阈值	提示精简后继续
429	RUN_QUEUED_DUE_TO_CAPACITY	并发阈值触发，任务进入排队	提示排队状态；已完成快照仍可看
502	CTS_PARAM_ANOMALY	上游返回 data: null 等参数/分页异常	明确提示“参数异常”，不能显示为 0 结果
504	AI_TIMEOUT	AI 草案或 AI 简历分析超过 10 秒	保留手工流程，允许稍后重试
504	CTS_TIMEOUT	上游页抓取超时	保留成功页；当前页允许重试

7. 工作流与控制流（Temporal）

Temporal 在本方案中不是“所有异步的筐”，而是专门负责长流程、可重试、可恢复、可暂停继续的编排层。所有不确定性 I/O 都必须下沉到 Activity；Workflow 只负责流程状态机、重试策略和补偿路径。

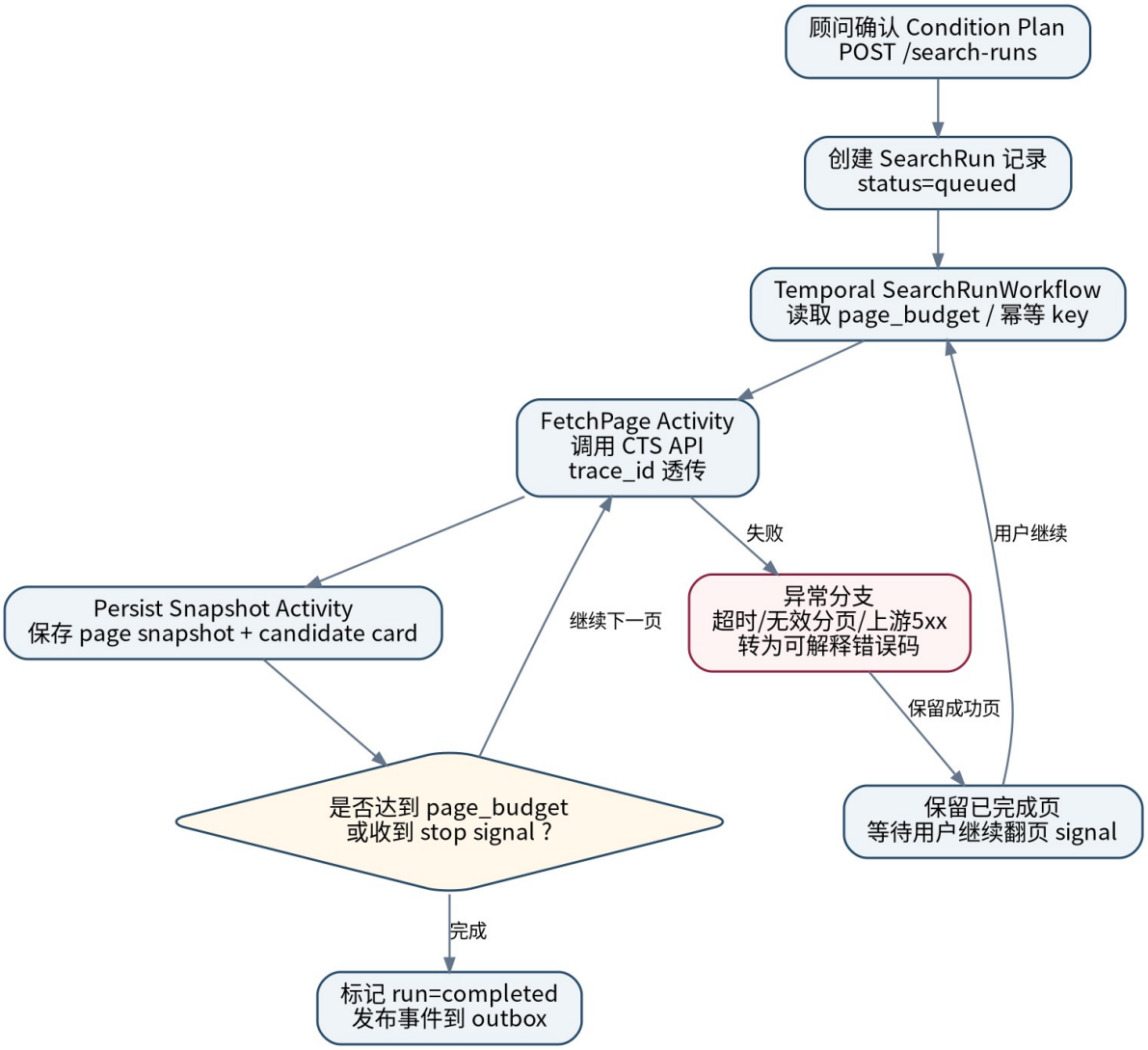

图 5 Search Run 工作流：快照优先、保留成功页、失败可解释、继续翻页由用户显式触发。

7.1 Workflow 清单

Workflow	输入	输出	说明
KeywordDraftWorkflow	caseId, jdVersionId, modelVersion, promptVersion	keywordDraftJobId / result	若 JD 过长，先进入 Condense Activity；用户不确认则不进入 Search Run
SearchRunWorkflow	runId, caseId, planId, pageBudget	run 完成状态 + 页快照	执行页抓取循环，支持 stop / continue signal
ResumeAnalysisWorkflow	resumeSnapshotId, jdVersionId, modelVersion, promptVersion	analysisJobId / analysis result	失败不得阻断 verdict 提交
ExportWorkflow	exportJobId, caseId, maskPolicy, reason	file ref / status	受权限与审计约束
EvalRunWorkflow	evalRunId, suiteId, datasetId, targetVersion	metrics / blocking result	可离线批跑，也可 PR gate 触发

7.2 重试、补偿与幂等

场景	策略
CTS 页抓取失败	同一 pageNo 在单个 run 内最多有限次数自动重试；超过阈值后记录错误并保留已成功页
AI 草案 / AI 分析超时	最多一次短退避重试；仍失败则返回 AI_TIMEOUT，界面保留人工操作
Search Run 重复创建	要求 idempotency key；若 key 相同则返回已存在 run
页快照重复写入	unique(run_id, page_no) 保证幂等；Activity 必须按 upsert 语义实现
导出重复提交	export_job 使用 idempotency key；同 key 在 pending / running 状态下禁止重复生成
集成事件重复消费	outbox_event.dedupe_key + 消费者侧 processed marker 防重

8. 数据模型、索引、幂等与 Postgres 策略

在“尽可能 Postgres-only”的约束下，本设计采用“单库、多 schema、渐进式 CQRS”。写模型优先保持清晰和一致性；读性能通过投影表、物化视图或专门 read model 表优化，而不是立刻引入额外中间件。

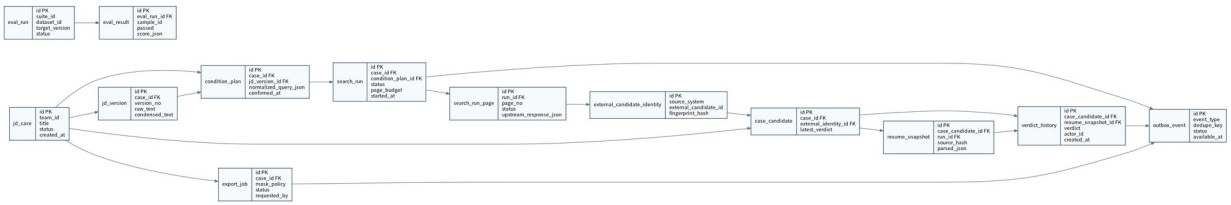

图 6 核心数据关系：Case / Plan / Run / Snapshot / Candidate / Verdict / Export / Eval。

8.1 PostgreSQL schema 建议

Schema	主要表
identity	user_account, team, role_binding
case_mgmt	jd_case, jd_version, keyword_draft_job, condition_plan
search	search_run, search_run_page, external_candidate_identity

Schema	主要表
review	case_candidate, resume_snapshot, resume_analysis_job, verdict_history, candidate_pool_entry
audit	export_job, audit_log
agentops	agent_run, workflow_health_projection, ai_version_metrics_daily
evals	eval_dataset, eval_sample, eval_suite, eval_run, eval_result
system	outbox_event, idempotency_record, schema_version_marker

8.2 关键表与索引

表	主键 / 关键唯一约束	关键索引	说明
search.search_run	PK(id)	(case_id, started_at desc), (status, started_at)	Case 视角历史与队列视角查询都要快
search.search_run_page	PK(id), UQ(run_id, page_no)	(run_id, page_no), (status, fetched_at)	幂等的页快照
review.case_candidate	PK(id), UQ(case_id, external_identity_id)	(case_id, latest_verdict), trigram(name_norm?)	Case 内候选聚合
review.resume_snapshot	PK(id), UQ(case_candidate_id, source_hash)	(case_candidate_id, created_at desc)	避免同一简历快照重复入库
review.verdict_history	PK(id)	(case_candidate_id, created_at desc), (actor_id, created_at desc)	复盘与审计
audit.audit_log	PK(id)	(actor_id, occurred_at desc), (target_type, target_id, occurred_at desc)	敏感动作审计查询
system.outbox_event	PK(id), UQ(dedupe_key)	(status, available_at)	轮询发布器的核心索引
evals.eval_run	PK(id)	(status, started_at desc), (target_version, suite_id)	比较不同版本结果
agentops.ai_version_metrics_daily	PK(day, model_version, prompt_version)	(day desc)	按版本展示命中率、超时率、完成率

8.3 JSONB 与结构化列的使用规则

- 来自外部 CTS 的原始响应、AI 输出详情、评测样本原始结果都可以存 JSONB，但关键过滤字段必须同步落到结构化列，不能让所有查询都在 JSONB 上完成。
- 典型规则：状态、时间、外键、角色、版本、布尔标志、数值评分必须结构化；原始 payload、证据片段、调试信息才允许放 JSONB。
- 如果某个 JSONB 字段开始进入高频筛选，必须升级为投影表或专门列，而不是继续依赖 ad-hoc 查询。

8.4 缓存策略

对象	策略
业务数据	MVP 不引入分布式缓存；直接读 PostgreSQL 投影表
prompt 模板 / ontology 索引 / docs 索引	允许进程内短 TTL 缓存（例如 60s-300s），丢失后可重建
Ops 看板聚合	优先做定时增量投影表，而不是每次实时扫明细
Eval 比较结果	使用 eval_run.summary_metrics 与 compare_view 投影，避免前端 N+1 聚合

9. 前端、Agent Ops、Agent Evals 平台设计

这里明确把运维监控可视化平台和评测平台纳入同一个产品工程体系，而不是后端私有工具。原因很简单：你的核心开发范式是 agent 并行编程，若没有统一的产品化 ops / evals 界面，系统的质量反馈会持续停留在工程师个人工作台里。

9.1 Angular shell 结构

路由域	对应 feature library	主要页面
/cases	features/cases	Case 列表、JD 编辑、关键词面板、Run 详情、候选详情
/agents	features/agents	agent run 列表、playbook、审计详情、失败修复入口
/ops	features/ops	队列概览、失败分类、超时分布、版本指标、工作流健康
/evals	features/evals	suite 列表、dataset 列表、发起评测、结果对比、样本 diff

- 所有 API client 与 DTO 从 OpenAPI 生成到 libs/ts/api-client-generated；feature 层不得手写跨边界 DTO。

- 前端状态管理优先保持显式与轻量。MVP 允许按页面局部状态 + query cache 方案，不在 Day 0 引入过重的全局状态框架。
- 长任务状态（run / export / eval）MVP 使用轮询即可；只有当负载和实时性证明需要时再升级 SSE。

9.2 Agent Ops 平台（新增基线）

页面	核心问题	关键数据源
队列总览	现在有哪些任务在排队、运行、失败、超时	workflow_health_projection
失败归因	到底是检索慢、AI 慢、导出慢还是上游异常	failure_reason_projection + trace links
版本指标	某个 prompt / model 版本近期的命中率、超时率、完成率怎样	ai_version_metrics_daily
工作流明细	某次 Search Run / Eval Run 卡在哪个 activity	Temporal history + product projection
Agent Run 审计	哪个 agent 改了什么、跑了哪些闸门、为什么失败	agent_run

> **注意**
> Product Ops 页面不是 Grafana 的替代品。它负责“可读的产品化概览与回溯入口”；真正的 metrics/logs/traces 深挖仍在 Grafana / Temporal UI 完成，并从产品页面提供深链跳转。

9.3 Agent Evals 平台（新增基线）

对象	定义	最小能力
Dataset	一组带版本的评测样本	导入、版本化、标签过滤、样本抽样
Suite	一组评分规则与阈值	配置 blocking / non-blocking 指标
Eval Run	某版本在某套件和某数据集上的执行结果	发起、查询、对比 baseline
Eval Result	单样本结果	看输入、输出、证据、分数、是否通过
Comparison View	两个版本的差异	看回归样本、分数变化、阻断原因

9.4 评测套件分类

Suite 类型	阻断级别	示例指标
Contract / Schema	P0 阻断	输出 schema 合法率、必填字段完整率、错误码映射正确率
Safety / Compliance	P0 阻断	联系方式泄露率、越权显示率、未授权敏感导出率必须为 0
Resilience / Degrade	P0 阻断	AI_TIMEOUT 后手工流程可继续率、上游 data:null 判错率
Product Quality	P1 观察 / 可配置阻断	关键词草案采纳率 proxy、证据覆盖率、verdict 辅助准确度
Latency / Cost	P1 观察	AI 平均时延、超时率、每 Run 成本估算

10. CI/CD、质量闸门、自建 runner 与安全

AI-Native 项目真正的 harness 不是 prompt 文本，而是“任何变更都必须穿过哪些机械闸门”。如果没有闸门，agent 能力越强，仓库退化越快。

图 7 PR 阶段强闸门：不是为了拖慢速度，而是为了让并行速度不以质量为代价。

10.1 PR 阶段闸门顺序

1. Markdown / docs lint：目录索引、交叉链接、必需文档存在性、计划文件完整性。
2. OWL 推理一致性 + SHACL 全量验证。
3. OpenAPI / AsyncAPI lint 与兼容性检查。
4. Codegen 执行并检查 generated 目录 diff 必须为 0。
5. Architecture tests：目录白名单、layer 依赖方向、禁止跨上下文导入。
6. Unit tests。
7. Integration tests：PostgreSQL、Temporal、CTS mock / fixture。
8. Critical eval suites：schema / safety / resilience。
9. 构建产物并等待合并；合并后再进入 CD。

10.2 自建 runner 策略

runner 组	用途	允许工作流	安全要求
build-pr	普通 PR 校验	lint / test / codegen / eval	尽量一次性或短命；不注入部署密钥

runner 组	用途	允许工作流	安全要求
trusted-merge	主干合并后构建	package / image build / release notes	只接收受信分支
deploy-sealos	部署到 Sealos	CD / rollout / smoke test	与 build runner 隔离，最小权限

- 私有仓库也不能掉以轻心。任何能从 fork 或 PR 执行不受信代码的人，都可能污染 self-hosted runner。
- 构建 runner 与部署 runner 必须隔离；前者不持有云端部署凭据，后者不接受来自不受信 PR 的作业。
- 如果条件允许，优先使用临时化 / 一次性 runner；日志要转储到外部存储，不能随 runner 生命周期消失。
- runner 镜像和注册脚本必须有版本更新策略；仓库要监控 GitHub 对 self-hosted runner 的版本要求变化。

10.3 机密与发布

对象	策略
tenant_secret / API 密钥	只存在于受信部署环境；PR 构建不注入
Sealos 部署凭据	只允许 deploy runner 使用；建议短时令牌或 OIDC
导出文件与评测工件	带权限标签；下载动作进入审计
GitHub Actions 第三方 action	固定到 commit SHA 或受信发行方，不接受随意 tag 漂移

11. 非功能实现、降级、审计与合规

11.1 性能与容量目标（工程实现口径）

目标	实现方式
Case / JD / 条件保存类接口 p95 < 300ms	纯同步 CRUD，直接命中写库或轻量投影
候选详情 / 历史 Run 查看类接口 p95 < 500ms	专门读投影、必要索引、避免实时聚合
Ops / Evals 列表与对比页 p95 < 1.5s	预聚合 projection / summary 表，不在线扫原始事件
AI 草案与 AI 简历分析硬超时 10s	Activity 超时 + 前端降级，不阻断后续人工流程
Search Run 并发达到阈值时进入排队	Temporal task queue + 429/queued 状态返回；已有快照继续可见

11.2 降级规则

异常	系统行为
AI 关键词草案超时	不自动发起 Search Run；保留可编辑条件面板；任务状态记为 timeout
JD 过长	阻断 AI，保留原文，要求用户确认精简文本后再进入 AI
上游分页异常 / data:null	标记为 CTS_PARAM_ANOMALY；明确提示参数异常，不显示为 0 结果
上游 5xx / timeout	保留成功页，当前页可重试；run 状态带失败摘要
AI 简历分析失败	详情页原文照常可读，verdict 可提交，候选池可操作

11.3 审计、权限与脱敏

要求	实现
字段级脱敏一致	在 presenter / export pipeline 共用同一份 field policy，避免列表、详情、导出、看板规则不一致
敏感操作全链路审计	查看联系方式、检索、verdict、导出、下载都写 audit_log
权限失败不泄露内容	仅返回 NO_CONTACT_PERMISSION 等明确错误，不展示局部号码或部分敏感字段
导出默认脱敏	非脱敏导出必须附带 reason 且通过角色 / 审批规则
保留复盘能力	Run、JD Version、verdict、导出任务、关键 AI 作业都带时间、版本、操作者与输入输出 hash

12. 实施节奏、Day 0 清单、风险与待确认问题

12.1 Day 0 / Week 1 落地顺序

时段	必须完成的结果
Day 0	仓库骨架、docs 骨架、ontology / contracts 初始版本、Makefile validate、CI skeleton、CODEOWNERS、runner 分组、AGENTS.md、issue 模板
Week 1 - Day 1	2-3 个核心概念 + 2 个事件 + 1 个 OpenAPI endpoint，全部可通过 OWL / SHACL / contract 校验
Week 1 - Day 2	codegen 跑通：生成 Python boundary models、TS

时段	必须完成的结果
	API client、docs/_generated 索引
Week 1 - Day 3	Search Run 最小主链路：Case -> Plan -> Run -> 1 页 CTS 抓取 -> Snapshot 落库 -> 候选列表可见
Week 1 - Day 4	agent registry、playbook、runs 审计、允许目录策略上线
Week 1 - Day 5	docs 与 Obsidian 图谱同步、Ops 首页最小可看、Eval 平台能跑一套 blocking suite

12.2 与 PRD 的交付映射

PRD 验收项 / 能力	本 TDD 对应实现
AC-01 核心闭环	Case & JD + Condition Plan + SearchRunWorkflow + Candidate Review + Verdict 持久化
AC-02 区分 0 结果与参数异常	CTS adapter 前后双层校验 + CTS_PARAM_ANOMALY 错误码
AC-03 AI 失败不阻塞	AI_TIMEOUT 降级策略 + Workflow 与 UI 规则
AC-04 中断后可恢复	page snapshot 幂等落库 + run 重试不生成重复页
AC-05 可完整复盘	JD Version / Search Run / Snapshot / Verdict / Audit 全链路保存
AC-06 导出默认脱敏	field policy + export workflow + 审批 / reason
AC-07 敏感动作审计	audit_log + 权限模型 + 查询 API
AC-08 Team Lead 看板	ops_summary_view + metrics projection
OP-01 / OP-02 监控与版本指标	Agent Ops 平台 + ai_version_metrics_daily
EV-01 内部评测页	Agent Evals 平台 + critical / non-critical suites

12.3 主要风险与缓解

风险	缓解策略
并行 agent 导致语义漂移	本体与合同先行；agent 只读 context pack；generated diff 必须为 0
流程复杂度爆炸	坚持模块化单体与渐进式 CQRS；Temporal 只用于长流程

风险	缓解策略
Postgres 压力过早上升	优先索引与投影优化；达到阈值后再考虑读库或外部基础设施
自建 runner 受不受信代码污染	runner 分组、最小权限、受信 / 非受信工作流隔离、短命 runner
Obsidian 成为第二套过时文档系统	docs 单一事实源 + 生成索引 + doc-gardener
Eval 平台沦为装饰性页面	把 schema / safety / resilience suites 直接接入 PR gate

12.4 待确认问题

- 外部简历库是否提供稳定候选唯一标识；若不稳定，需确认“疑似重复”体验与指纹策略可接受性。
- 敏感字段列表、不同角色可见边界、非脱敏导出审批规则尚需业务 / 合规最终确认。
- 是否提供可用的对象存储承载大体积导出与评测工件；若没有，MVP 需接受数据库膨胀风险或临时文件方案。
- 认证是否直接接入企业 SSO / OIDC；若不是，MVP 账号体系如何初始化。
- 模型预算上限与允许模型范围；这会影响 prompt 策略、并发阈值和 eval 基线。

附录 A. 错误码字典（建议初版）

代码	适用边界	阻断级别
INVALID_PAGINATION_PARAMS	platform-api	P0
PLAN_NOT_CONFIRMED	platform-api	P0
JD_TOO_LONG	keyword draft	P0
AI_TIMEOUT	keyword draft / resume analysis	P0
CTS_TIMEOUT	search workflow	P0
CTS_PARAM_ANOMALY	search workflow	P0
NO_CONTACT_PERMISSION	details / export	P0
IDEMPOTENCY_CONFLICT	search run / export / eval run	P0
EVAL_BLOCKING_FAILED	CI / release gate	P0

附录 B. 推荐 AGENTS.md 内容骨架

```
# AGENTS.md
1. 这是什么仓库：产品、部署单元、最小运行方式。
2. 先看哪里：docs/00-INDEX.md、docs/ARCHITECTURE/context-map.md、docs/CONTRACTS/index.md。
```

```
3. 严禁事项：
   - 不得绕过 contracts/ 直接改边界 DTO
   - 不得在 services/*/domain 中引入 infra 依赖
   - 不得手改 docs/_generated 或 libs/*/generated
4. 如何执行：make validate / make test / make eval-critical
5. 如何领取任务：先读 issues/context-packs/<issue-key>.md
6. 允许目录：以 issue context pack 为准
7. 失败怎么办：只能修代码使其符合现有规则，或提交本体/合同变更 PR，禁止临时放宽规则
```

附录 C. 参考资料与输入基线

- 输入文档：prd.pdf（PRD v1.0）；基于 Clean Architecture + DDD + EDA + CQRS，并以本体论建模约束 AI 代码生成的 AI-Native Harness Engineering 落地研究报告.pdf；面向猎头的“根据 JD 寻找匹配简历”MVP 系统深度研究与架构设计报告.md。
- OpenAI: Harness engineering: leveraging Codex in an agent-first world（用于 docs as system of record、AGENTS.md 目录化、exec plan、doc-gardening 的工程原则参考）。
- GitHub Docs / Changelog：self-hosted runner 安全与版本更新要求。